RBFL con habilitación entrenable y realimentación

Una victoria medida en paridad acumulativa y sus límites

Estudio reproducible del repositorio BFL

10 de octubre de 2026

Resumen

Presentamos una red recurrente de ocho MUX booleanos con conexiones discretas, estado registrado y una ruta entrenable de habilitación por unidad. Su entrenamiento conserva el criterio *argmin* lexicográfico del BFL binary-global: minimiza primero los errores totales, luego los errores nuevos en predicciones antes correctas y finalmente el número de bits de configuración cambiados. En cinco semillas de una tarea de paridad acumulativa, la variante `rbfl_used_m_trainable_feedback` alcanzó 100 % de exactitud por bit y por secuencia en 400 secuencias de longitudes no vistas y otro 100 % en 400 secuencias con distribución desplazada. La RNN de Elman de control alcanzó 81,08 % ± 19,31 % por bit en el test de longitudes nuevas. La ablación con habilitación entrenable pero sin persistencia temporal alcanzó 54,92 % ± 1,61 %. El éxito es específico de esta construcción: en la tarea estática histórica, la variante RBFL evaluada obtuvo 73,44 %, por debajo del 90,00 % del BFL binary-global y del 99,29 % de la MLP. Publicamos código, particiones deterministas, resultados por semilla y límites de la comparación.

1. Alcance y procedencia

El resultado de referencia del BFL binary-global es 90,3066 % de exactitud media de test en 50 semillas, archivado en el commit `53c2f46`. Recompilamos sin cambios el benchmark de ese commit: en 20 semillas produjo 90,6132 % ± 1,6145 %. El modelo original usa 32 rutas fijadas por cinco bits de entrada, 16 características booleanas y una mayoría de características activas. Su entrenamiento enumera propuestas de hasta tres flips, las enmascara con las características `USED` del ejemplo actual y evalúa el grupo completo de entrenamiento correspondiente a esa ruta. Compara (ERR, FORGET, CHANGE_COST) lexicográficamente. En ese código, `CHANGE` designa la propuesta de configuración; no es la máscara de error.

El objetivo de este trabajo es evaluar una arquitectura distinta, con memoria temporal explícita. Por ello, *variante ganadora* significa exclusivamente la mejor RBFL medida en la tarea temporal de paridad de este protocolo; no implica una superioridad en tareas estáticas ni en datos reales. Los identificadores de las filas son:

- `rbfl_used_m_trainable_feedback`: variante ganadora en paridad;
- `rbfl_used_m_fixed1_feedback`: control con habilitación fija;
- `rbfl_used_m_trainable_no_feedback`: ablación de memoria.

2. Arquitectura RBFL

El estado tiene ocho bits, $H_t = (Q_{1,t}, \ldots, Q_{8,t})$. Cada entrada $A_{i,t}$ y $B_{i,t}$ selecciona una de veinte fuentes: las constantes 0 y 1, los diez bits de X_t o los ocho bits de H_t. Todas las unidades leen el mismo H_t y luego se escribe H_{t+1}, sin ciclos combinacionales. Para cada MUX,

$$M_{i,t} = (A_{i,t} \wedge \neg Q_{i,t}) \vee (B_{i,t} \wedge Q_{i,t}), \tag{1}$$

$$Q_{i,t+1} = Q_{i,t} \oplus \left[\mathrm{USED}_{M,i,t} \wedge (Q_{i,t} \oplus M_{i,t})\right]. \tag{2}$$

En la variante ganadora, $\mathrm{USED}_{M,i,t}$ también selecciona una fuente booleana entre esas veinte. Su valor no depende del target en inferencia. La salida es un bit seleccionado de H_{t+1}, opcionalmente invertido. El estado inicial es cero y se reinicia entre secuencias, conservándose entre pasos de una misma secuencia.

Tabla 1. MUX retroalimentado antes de la habilitación.

A	B	M	A	B	M
0	0	0	0	1	Q
1	0	$\neg Q$	1	1	1

La máscara de uso registra $U^A_{i,t} = \mathrm{USED}_{M,i,t} \wedge \neg Q_{i,t}$ y $U^B_{i,t} = \mathrm{USED}_{M,i,t} \wedge Q_{i,t}$. Se agregan estas observaciones en USED_G sobre las trayectorias de entrenamiento. El buscador evalúa también rewires de rutas inactivas: de otro modo, una ruta nunca usada no podría descubrirse. Cada índice de fuente ocupa cinco bits; el costo de un rewire es la distancia de Hamming entre el índice viejo y el nuevo. La variante ganadora tiene 124 bits de configuración entrenables y ocupa 44 bytes como objeto C++ medido.

3. Greedy lexicográfico

Definimos $\mathrm{CHANGE}_{\mathrm{ERROR}} = P_\theta(X) \oplus Y$, separado de la propuesta de cambio de parámetros Δ. Para cada candidato se reinicia H_0, se reproduce *toda* la colección de secuencias de entrenamiento y se obtiene $E_\Delta = Y \oplus P_{\theta_\Delta}(X)$. Con $E_0 = Y \oplus P_\theta(X)$, el criterio implementado es

$$\Delta^* = \arg\min_{\Delta}^{\mathrm{lex}} \left(\sum E_\Delta,\ \sum (\neg E_0 \wedge E_\Delta),\ \mathrm{popcount}(\theta \oplus \theta_\Delta) \right). \tag{3}$$

El modelo actual se compara con la tupla $(\sum E_0, 0, 0)$; una propuesta se conserva solo si mejora esa tupla y, en caso contrario, se revierte. No hay gradientes, SGD, Adam ni pesos flotantes dentro de RBFL. La búsqueda prueba un rewire completo o una modificación de salida o inicialización por candidato, con hasta cuatro iteraciones greedy. Esta vecindad es distinta de los flips de hasta tres características del BFL histórico. `FORGET` solo desempata después de igualdad en errores totales.

4. Protocolo experimental

El benchmark estático histórico contiene los 1024 vectores binarios distintos de diez bits, barajados con `std::mt19937(seed)` y divididos en 600 de entrenamiento y 424 de test. Una segunda partición del mismo dominio reserva 600/200/224 para entrenamiento/validación/test. Se conservó el BFL binary-global del commit histórico y una MLP de 16 unidades ocultas con 193 parámetros `float64`, 500 épocas de Adam y tasa 0,03. La RNN de Elman de control tiene ocho unidades ocultas y 161 parámetros `float64`; usa BPTT completo, Adam, 500 épocas y tasa 0,01.

En la tarea temporal, el bit relevante $x_{t,0}$ es Bernoulli justo y los otros nueve bits son distractores. El target es $y_t = \bigoplus_{s=0}^{t} x_{s,0}$. Por semilla se generan 160 secuencias de entrenamiento y 40 de validación, de longitudes 4–8. El test final tiene 80 secuencias de longitudes 9–12 y otro test independiente de 80 secuencias de esas longitudes con $\Pr(x_{t,0} = 1) = 0,2$. Son cinco semillas. El test temporal final usa un flujo aleatorio separado, fijado después de la exploración preliminar y antes de obtener estas cifras. No se optimizan parámetros RBFL con datos del test.

Las métricas de inferencia excluyen lectura y tokenización: se hacen 10 000 llamadas de warmup y cinco lotes de 100 000 llamadas, cada una para una secuencia completa. Los nanosegundos por paso dividen el tiempo por longitud media. La memoria persistente es el objeto del modelo; la memoria temporal es una cota inferior de los datos de inferencia, sin metadatos del asignador. Las mediciones usan `g++ 13.3`, `-O3 -march=native`, en AMD Ryzen 7 5700U. No había contadores de instrucciones CPU accesibles.

5. Resultados

Tabla 2. Tarea estática histórica, media de cinco semillas; RBFL con $\mathrm{USED}_M = 1$.

Modelo	Train	Test ± DE	Entrenamiento (ms)	Inferencia (ns/paso)
BFL binary-global	96,13 %	90,00 % ± 1,82	242,19	14,13
RBFL, $\mathrm{USED}_M = 1$	76,27 %	73,44 % ± 1,19	18,88	45,78
MLP, 16 ocultas	100,00 %	99,29 % ± 0,47	160,65	425,32
RNN de Elman, 8 ocultas	99,83 %	97,31 % ± 1,40	110,40	347,15

La partición estática 600/200/224 confirma el resultado negativo: test 90,63 % para BFL, 73,39 % para RBFL con $\mathrm{USED}_M = 1$, 99,38 % para la MLP y 97,23 % para la RNN. Por tanto, el resultado temporal favorable no puede extrapolarse a una mejora general frente al BFL original.

Tabla 3. Paridad acumulativa, media de cinco semillas en longitudes nuevas.

Modelo	Test bit ± DE	Exacto	Dist. nueva	Entren. ms
MLP sin memoria	50,56 % ± 2,23	0,25 %	52,04 %	224,89
RNN de Elman	81,08 % ± 19,31	54,75 %	80,79 %	148,85
RBFL, $\mathrm{USED}_M = 1$	58,82 % ± 0,96	8,75 %	64,94 %	38,08
RBFL, USED_M **aprend. + memoria**	**100,00 % ± 0,00**	**100,00 %**	**100,00 %**	57,37
RBFL, USED_M aprend., sin memoria	54,92 % ± 1,61	0,25 %	59,88 %	34,87

Tabla 4. Costo de inferencia temporal; bytes temporales como cota inferior.

Modelo	Modelo (B)	Temporal (B)	Total (B)	ns/paso
MLP sin memoria	1544	128	1672	393,05
RNN de Elman	1288	940	2228	276,51
RBFL, $\text{USED}_M = 1$	44	116	160	29,70
RBFL, USED_M **entrenable + feedback**	44	116	160	31,88
RBFL, USED_M entrenable, sin feedback	44	116	160	31,22

La variante ganadora mejora 18,92 puntos porcentuales sobre la media de la RNN en este test de paridad, pero la RNN presenta alta variación entre semillas. Frente a la misma RBFL sin persistencia, la diferencia es 45,08 puntos: la realimentación aporta una ventaja medible bajo este protocolo. La ablación que quita `FORGET` del control $\text{USED}_M = 1$ no alteró la accuracy en estas cinco semillas; tampoco lo hizo permitir Q_0 entrenable en esa vecindad. El detalle por semilla está en `results/ablation.csv`.

6. Discusión y límites

La tarea favorece una solución booleana compacta: una habilitación dependiente de la entrada puede realizar un toggle condicional de Q. Eso explica un mecanismo plausible, pero no demuestra que la arquitectura tenga ventaja en otros problemas de memoria. Se midieron cinco semillas y una sola regla temporal sintética; no se probaron secuencias ruidosas, datos reales ni una búsqueda exhaustiva de hiperparámetros. Las arquitecturas no comparten presupuesto de parámetros ni tiempo de entrenamiento: la comparación de costos caracteriza estas implementaciones concretas. La RNN tiene gradientes y mayor capacidad nominal, mientras que RBFL enfrenta una búsqueda discreta limitada a un cambio por iteración. Las cifras de memoria omiten gastos del asignador; las de tiempo dependen de la máquina y del código. El BFL histórico no tiene un estado temporal comparable y por ello no es un control de paridad con memoria.

7. Resultados individuales y verificaciones

Tabla 5. Exactitud por bit en longitudes nuevas, por semilla.

Semilla	RBFL ganadora	RNN de Elman	RBFL sin memoria
1	100,00 %	99,05 %	57,35 %
2	100,00 %	54,96 %	54,37 %
3	100,00 %	93,76 %	52,46 %
4	100,00 %	97,35 %	54,70 %
5	100,00 %	60,26 %	55,70 %

Estos valores explican la elevada dispersión de la RNN: tres semillas superaron el 93 % y dos quedaron cerca del azar. La tabla no sustituye un estudio con más semillas y tareas. En las pruebas de implementación se comprobaron las cuatro filas de la tabla de verdad para ambos valores de Q, la actualización simultánea entre unidades, la retención con $\text{USED}_M = 0$, la persistencia temporal, el registro de las rutas A y B seleccionadas, y la igualdad $\text{CHANGE}_{\text{ERROR}} = P \oplus Y$. Una prueba de entrenamiento verificó que una propuesta útil cambia parámetros y que ninguna modificación de costo positivo se acepta cuando el error ya es cero. Las 120 filas de la traza final cumplen $\text{ERR}_{\text{mejor}} \leq \text{ERR}_{\text{anterior}}$. El ejecutable de inferencia RBFL recibe únicamente las entradas: las etiquetas aparecen solo en la evaluación supervisada.

8. Reproducción

El código y los datos están en el directorio de investigación `research/rbfl_vs_ann/`. Allí, el comando de reproducción es:

```
bash scripts/reproduce.sh 5 20
```

Este comando recompila los cuatro modelos, verifica RBFL y genera los CSV de resultados, resúmenes, ablaciones y trazas. El PDF se regenera con `bash scripts/build_paper.sh`. Las etiquetas de variantes fueron aclaradas después de medir; las particiones inexistentes figuran como `NA` en los CSV, conservando todas las mediciones. El identificador de la variante ganadora es:

`rbfl_used_m_trainable_feedback`.